

What is Refactoring? Why do we refactor our code? 3

Define code smell. Name 10 different common code smells. 4

Find out the smells from the following code. Mention the name of the refactoring process to improve this code segment. Write the refactored code. 5

```
String foundPerson(String[] people){
    for (int i = 0; i < people.length; i++) {
        if (people[i].equals ("Don")){
            return "Don";
        }
        if (people[i].equals ("John")){
            return "John";
        }
        if (people[i].equals ("Kent")){
            return "Kent";
        }
    }
    return "";
}
```

What is Refused Bequest? Give one example of Refused Bequest and refactor it. 5

What does Extract Method means? Which code smells are removed using this process? 3

a)

Refactoring is a series of small steps, each of which changes the program's internal structure without changing its external behavior.

Why do we Refactor?

- ☑ Helps us deliver **more business value faster**
- ☑ Improves the **design** of our software
 - ☑ Combat's "bit rot"
 - ☑ Easier to maintain and understand
 - ☑ Easier to facilitate change
 - ☑ More flexibility
 - ☑ Increased re-usability

Why do we Refactor?...

- ☑ Minimizes *technical debt*
- ☑ Keep **development** at *speed*
- ☑ To make the software easier to **understand**
 - ☑ Write for people, not the compiler
 - ☑ Understand unfamiliar code
- ☑ To help find **bugs**
 - ☑ refactor while debugging to clarify the code
- ☑ To "*Fix broken windows*" - Pragmatic Programmers

b)

A code smell is a design that duplicates, complicates, bloats or tightly couples code

Common Code Smells

- | | |
|--|--|
| ☒ Inappropriate Naming | ☒ Long Method |
| ☒ Comments | ☒ Long Parameter List |
| ☒ Dead Code | ☒ Switch Statements |
| ☒ Duplicated code | ☒ Speculative Generality |
| ☒ Primitive Obsession | ☒ Oddball Solution |
| ☒ Large Class | ☒ Feature Envy |
| ☒ Lazy Class | ☒ Refused Bequest |
| Alternative Class with
Different Interface | ☒ Black Sheep |
-

c)

Odd Ball Solution and switch-case smell. Refactored using substitute algorithm:

```
import java.util.ArrayList;
import java.util.List;

String foundPerson(String[] people) {
    List<String> targetNames = new ArrayList<>();
    targetNames.add("Don");
    targetNames.add("John");
    targetNames.add("Kent");

    for (String person : people) {
        if (targetNames.contains(person)) {
            return person;
        }
    }
    return "";
}
```

d)

This rather potent odor results when subclasses inherit code that they don't want. In some cases, a subclass may "refuse the bequest" by providing a do-nothing implementation of an inherited method.

Example-

```
1  class Employee {
2      void work() {
3          System.out.println("Working on tasks...");
4      }
5
6      void attendMeeting() {
7          System.out.println("Attending a meeting...");
8      }
9  }
10
11 class Intern extends Employee {
12     @Override
13     void attendMeeting() {
14         throw new UnsupportedOperationException("Interns are not allowed in meetings!");
15     }
16 }
17
18 public class Main {
19     public static void main(String[] args) {
20         Employee intern = new Intern();
21         intern.attendMeeting(); // Throws exception, bad design
22     }
23 }
24
```

Refactor:

```
1  class Employee {
2      void work() {
3          System.out.println("Working on tasks...");
4      }
5  }
6
7  class Manager extends Employee {
8      void attendMeeting() {
9          System.out.println("Attending a meeting...");
10     }
11 }
12
13 class Intern extends Employee {
14     // No `attendMeeting()` method, avoiding refused bequest
15 }
16
17 public class Main {
18     public static void main(String[] args) {
19         Employee intern = new Intern();
20         intern.work(); // Works fine
21
22         Manager manager = new Manager();
23         manager.attendMeeting(); // Works fine
24     }
25 }
```

e)

- **Definition:** Extract Method is a refactoring technique where you take a block of code and turn it into its own method.
- **Code Smells Removed:**
 - **Long Method:** Breaks down a long, complex method into smaller, more manageable methods.
 - **Duplicated Code:** If the extracted block of code is duplicated elsewhere, you can replace the duplication with a call to the new method.

2.	a.	"Comments are often used as deodorant" -- justify this statement. What are the remedies of comment?	4
	b.	What do you understand by semantic duplication? Give one example of such duplication?	3
	c.	What does Parameter Object means? Give one example of Parameter Object	3

a)

Same as of the 2023's

Go to 2023: [Design Pattern Final](#)

b)

Semantic Duplication:

Semantic duplication in code is a code smell where different parts of the code perform the same or very similar operations, even if they don't look exactly alike. It's like having two functions that calculate a discount, but one uses a **for** loop and the other uses a **map** function. Both achieve the same result, but the implementation is different.

1stLevel - For and For Each Loop	<pre>stack.push(1); stack.push(3); stack.push(5); stack.push(10); stack.push(15);</pre>
2ndLevel - Loop v/s Lines repeated	<pre>v/s for(int i : asList(1,3,5,10,15)) stack.push(i);</pre>

c)

Parameter Object:

A **Parameter Object** is a design pattern where multiple related parameters are encapsulated into a single object instead of passing them separately as arguments to a function or method.

Example:

```
public void createUserAccount(String name, String email, String password, Date dateOfBirth) {  
    // ...  
}  
  
public void createUserAccount(UserAccountDetails userAccountDetails) {  
    // ...  
}
```

d. What kind of code smells you get from the below code segment?

```
class Employee  
{  
  
    private int _type;  
    private const int ENGINEER = 0;  
    private const int SALESMAN = 1;  
    private const int MANAGER = 2;  
    private double _monthlySalary;  
    private double _commission;  
    private double _bonus;  
  
    public Employee(int type, double salary, double commission, double bonus)  
    {  
        _type = type;  
        _monthlySalary = salary;  
        _commission = commission;  
        _bonus = bonus;  
    }  
}
```

```

public double PayAmount()
{
    switch (_type)
    {
        case ENGINEER:
            return _monthlySalary;
        case SALESMAN:
            return _monthlySalary + _commission;
        case MANAGER:
            return _monthlySalary + _bonus;
        default:
            throw new Exception("Incorrect Employee");
    }
}

static void Main()
{
    Employee manager = new Employee(2, 20000, 5000, 0.25);
    manager.PayAmount();
    Employee salesman = new Employee(1, 10000, 2500, 0.15);
    salesman.PayAmount();
    Employee engineer = new Employee(0, 15000, 3000, 0.20);
    engineer.PayAmount();
}

```

Refactor the code and mention the approach of refactoring you applied.

Code Smells: Primitive obsession, Switch Statement, Refused Bequest and Inappropriate Naming.

```

// Abstract base class for Employee
abstract class Employee {
    protected double monthlySalary;

    public Employee(double salary) {
        this.monthlySalary = salary;
    }

    // Abstract method to calculate pay amount
    public abstract double payAmount();
}

// Engineer subclass
class Engineer extends Employee {
    public Engineer(double salary) {
        super(salary); // Engineers only need salary
    }

    @Override
    public double payAmount() {
        return monthlySalary; // Engineers only get their monthly salary
    }
}

```

```

// Salesman subclass
class Salesman extends Employee {
    private double commission; // Salesmen need commission

    public Salesman(double salary, double commission) {
        super(salary);
        this.commission = commission;
    }

    @Override
    public double payAmount() {
        return monthlySalary + commission; // Salesmen get salary + commission
    }
}

// Manager subclass
class Manager extends Employee {
    private double bonus; // Managers need bonus

    public Manager(double salary, double bonus) {
        super(salary);
        this.bonus = bonus;
    }

    @Override
    public double payAmount() {
        return monthlySalary + bonus; // Managers get salary + bonus
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        Employee engineer = new Engineer(15000);
        Employee salesman = new Salesman(10000, 2500);
        Employee manager = new Manager(30000, 0.25);

        System.out.println("Engineer's Pay: " + engineer.payAmount());
        System.out.println("Salesman's Pay: " + salesman.payAmount());
        System.out.println("Manager's Pay: " + manager.payAmount());
    }
}

```

c. What is Primitive Obsession? Give one example of primitive obsession and refactor it. What are the remedies of primitive obsession? 4

Design Pattern Final

3. What is the relation between patterns and refactoring? 2
What is "Feature Envy"? What are the remedies of this code smell? 4

a)

The relationship between **design patterns** and **refactoring** is deeply interconnected. Both are essential tools in software development, and they often work together to improve the design, maintainability, and flexibility of code. Here's how they relate:

1. Design Patterns Provide Target Structures for Refactoring

- **Design patterns** are well-established solutions to common software design problems. They provide a blueprint for organizing code in a way that promotes flexibility, reusability, and maintainability.
- **Refactoring** is the process of improving the internal structure of code without changing its external behavior. When refactoring, developers often aim to introduce design patterns to address specific design flaws or improve the overall architecture.
- For example:
 - If you notice a lot of duplicated code, you might refactor it to use the **Template Method Pattern**.
 - If you have a class with too many responsibilities, you might refactor it using the **Strategy Pattern** to separate concerns.

2. Refactoring Helps Apply Design Patterns Incrementally

- Design patterns are not always applied from the start of a project. As the codebase evolves, developers may identify areas where a design pattern would be beneficial.
- **Refactoring** allows developers to incrementally introduce design patterns into the codebase without disrupting the existing functionality.
- For example:

- If you have a large switch statement that violates the **Open/Closed Principle**, you can refactor it to use the **Factory Method Pattern** or **Strategy Pattern** to make the code more extensible.

3. Refactoring Can Reveal the Need for Design Patterns

- During refactoring, developers often uncover design flaws or areas where the code could be improved. These flaws might indicate the need for a design pattern.
- For example:
 - If you notice that a class is tightly coupled to multiple other classes, refactoring might reveal the need for the **Observer Pattern** to decouple the classes.
 - If you find that a class is doing too much, refactoring might suggest the use of the **Decorator Pattern** to add behavior dynamically.

b)

Feature Envy

- ☑ A method that seems more interested in some other class than the one it is in.
- ☑ Data and behavior that acts on that data belong together. When a method makes too many calls to other classes to obtain data or functionality, Feature Envy is in the air.
- ☑ Remedies:
 - ☑ Move Field
 - ☑ Move Method
 - ☑ Extract Method



178

- 1) What smells do you get from the following code? Refactor the code to remove the smells. 6
(Don't consider the comment as smell here. This comment actually says that there is some code that handles the IsValid method. This code is out of the scope of this question.)

```
Class Validator
{
    private bool result;
    public bool IsValid(object obj)
    {
        //in this method we have a code that can validate
        //string/int/double/float objects
        //result is set here using some code

        return result;
    }
}

Class ValidatorFactory
{
    public Validator CreateValidtor(string type)
    {
        if(type == "string" || type == "int" ||
            type=="double" || type=="float")
            return new Validator();

        return null;
    }
}

Class TestMainClass
{
    static void main(string[] args)
    {
        Validator validator =
            new ValidatorFactory().CreateValidtor(args[0]);

        if(validator != null)
            validator.IsValid(args[1]);
    }
}
```

The provided code has several **code smells** that can be refactored to improve readability, maintainability, and flexibility. Below is an analysis of the smells and a refactored version of the code.

Code Smells:

1. **Primitive Obsession:**

- The CreateValidator method uses a string (type) to determine the type of validator to create. This is error-prone and not type-safe.

2. Violation of the Open/Closed Principle:

- The Validator class handles validation for multiple types (string, int, double, float) in a single method. This makes the class rigid and hard to extend for new types.

3. Magic Strings:

- The strings "string", "int", "double", and "float" are hardcoded in the CreateValidator method. These are magic strings that can lead to errors if misspelled.

4. Poor Encapsulation:

- The Validator class has a result field that is not encapsulated properly. It is modified internally but exposed publicly through the IsValid method.

5. Null Return:

- The CreateValidator method returns null if the type is not recognized. This can lead to NullReferenceException if not handled properly.

6. Inconsistent Naming:

- The class name TestMainClass is not descriptive, and the method main should follow C# naming conventions (Main).

Refactored Code:

To address these smells, we can:

1. Use **polymorphism** to handle different types of validators.
2. Replace magic strings with an **enum**.
3. Ensure the **Open/Closed Principle** is followed by making the code extensible.
4. Improve encapsulation and naming.

```
// Base Validator class
abstract class Validator {
    public abstract boolean isValid(Object obj);
}

// StringValidator subclass
class StringValidator extends Validator {
    @Override
    public boolean isValid(Object obj) {
        return obj instanceof String && obj != null;
    }
}
```

```
// IntValidator subclass
class IntValidator extends Validator {
    @Override
    public boolean isValid(Object obj) {
        try {
            Integer.parseInt(obj.toString());
            return true;
        } catch (NumberFormatException e) {
            return false;
        }
    }
}

// FloatValidator subclass
class FloatValidator extends Validator {
    @Override
    public boolean isValid(Object obj) {
        try {
            Float.parseFloat(obj.toString());
            return true;
        } catch (NumberFormatException e) {
            return false;
        }
    }
}
```

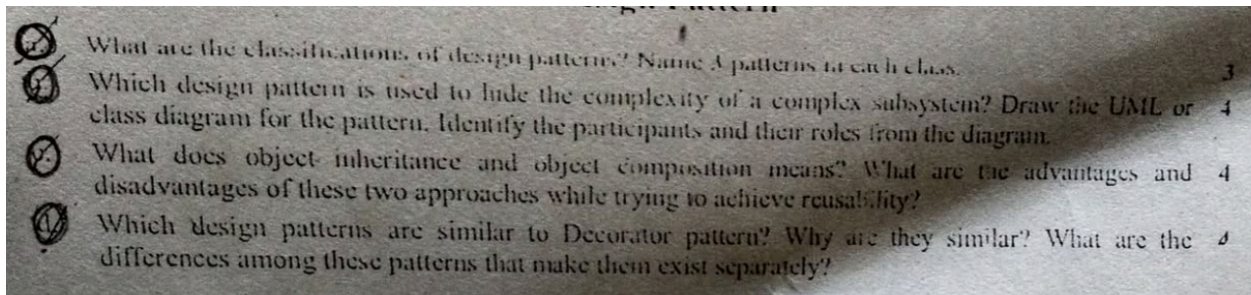
```
// DoubleValidator subclass
class DoubleValidator extends Validator {
    @Override
    public boolean isValid(Object obj) {
        try {
            Double.parseDouble(obj.toString());
            return true;
        } catch (NumberFormatException e) {
            return false;
        }
    }
}

// Validator factory class
class ValidatorFactory {
    public static Validator createValidator(String type) {
        switch (type.toLowerCase()) {
            case "string":
                return new StringValidator();
            case "int":
                return new IntValidator();
            case "float":
                return new FloatValidator();
            case "double":
                return new DoubleValidator();
            default:
                return null;
        }
    }
}
```



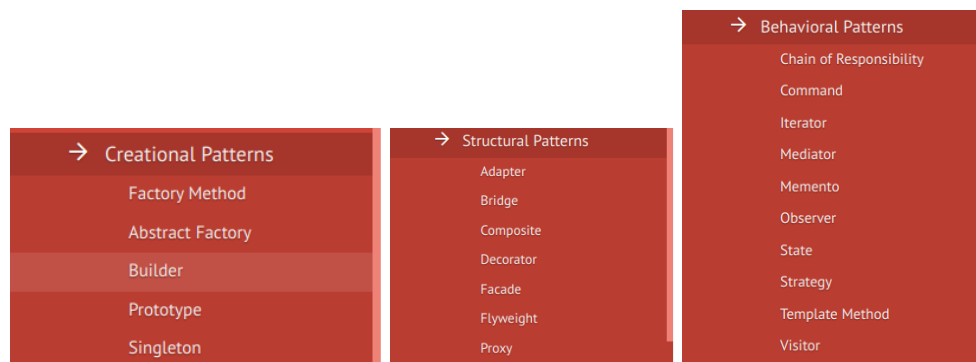
```
// Test Class
public class TestMainClass {
    public static void main(String[] args) {
        if (args.length < 2) {
            System.out.println("Usage: java TestMainClass <type> <value>");
            return;
        }

        Validator validator = ValidatorFactory.createValidator(args[0]);
        if (validator != null) {
            System.out.println("Validation result: " + validator.isValid(args[1]));
        } else {
            System.out.println("Error: Invalid type " + args[0]);
        }
    }
}
```



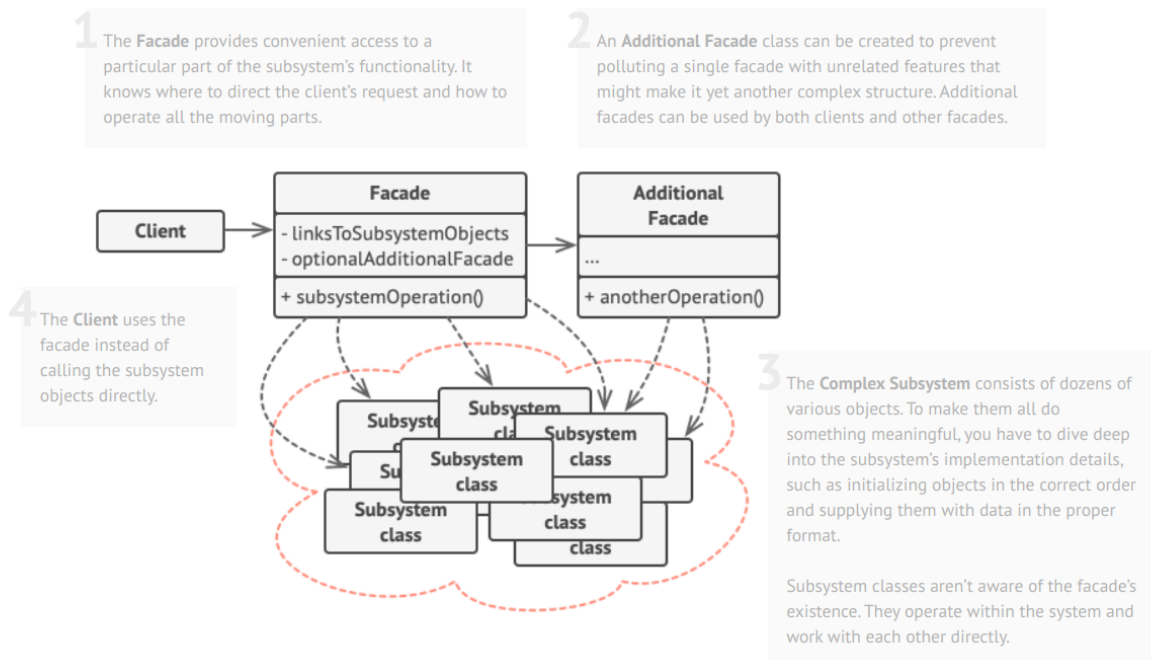
a)

There are three types of design pattern- Creational, Structural and Behavioral



b)

Facade.



c)

Object Inheritance (IS-A Relationship)

Inheritance allows a subclass to derive properties and behavior from a superclass. The subclass automatically gains access to the superclass's methods and attributes, enabling code reuse.

Advantages of Inheritance:

- **Code reusability:** Common functionality is defined once in the superclass and shared across subclasses.
- **Easier extension:** New features added to the superclass are available to all subclasses.
- **Supports polymorphism:** A subclass can be used wherever a superclass object is expected.

Disadvantages of Inheritance:

- **Tight coupling:** Subclasses depend on the superclass, so changes in the superclass can affect all subclasses.

- Inflexibility: A subclass inherits all behaviors from the superclass, even if it only needs a part of them.
 - Increased complexity: Deep inheritance hierarchies can be difficult to manage and understand.
-

Object Composition (HAS-A Relationship)

Composition means an object contains another object instead of inheriting from it. Instead of extending a class, it delegates behavior to the contained object.

Advantages of Composition:

- Loose coupling: Changes in one class do not directly affect others, making the system more flexible.
- Better maintainability: Individual components can be modified or replaced independently.
- Higher reusability: A class can contain multiple independent components instead of being tightly bound to a hierarchy.

Disadvantages of Composition:

- More boilerplate code: Requires extra methods to delegate behavior to the contained objects.
- Indirection: Behavior is not inherited directly, requiring additional method calls to access features.

d)

Pattern	Similarities with Decorator	Key Differences	Use Cases
Adapter	- Wraps an existing object. - Follows the Single Responsibility Principle.	- Converts an interface into another expected interface. - Used for compatibility rather than extending behavior.	- Integrating two incompatible systems. - Making legacy code work with new interfaces.
Proxy	- Uses a wrapper around the original object. - Implements the same interface as the wrapped object.	- Controls access to an object (security, caching, lazy initialization). - Focuses on restricting or managing access, not extending functionality.	- Managing resource-intensive operations. - Adding security layers or caching mechanisms.
Composite	- Relies on object composition. - Allows treating individual and wrapped objects uniformly.	- Structures objects into a tree hierarchy (whole-part relationships). - Used for organizing objects rather than modifying them.	- Representing UI components, file systems, or organizational hierarchies. - Handling complex structures where groups and individuals must be treated the same.

What is the motivation of Abstract Factory pattern? What are the participant objects in Abstract Factory pattern? Briefly explain the role of each participant object. Show an example code that demonstrates the use of Abstract Factory pattern.

The **Abstract Factory Pattern** is used when a system needs to create families of related objects **without specifying their concrete classes**. It helps maintain **encapsulation and flexibility** by delegating the object creation process to factory classes, making it easy to switch between different implementations.

Participant	Role
AbstractFactory	Declares an interface for creating families of related objects.
ConcreteFactory	Implements the creation methods for a specific product family.
AbstractProduct	Defines an interface for a type of product.
ConcreteProduct	Implements the AbstractProduct interface with a specific variant.
Client	Uses the factory interface to create products without knowing the concrete classes.

```
1  // AbstractProduct: Chair
2  interface Chair {
3      void sitOn();
4  }
5
6  // AbstractProduct: Sofa
7  interface Sofa {
8      void lieOn();
9  }
10
11 // ConcreteProduct: ModernChair
12 class ModernChair implements Chair {
13     @Override
14     public void sitOn() {
15         System.out.println("Sitting on a modern chair.");
16     }
17 }
18
19 // ConcreteProduct: VictorianChair
20 class VictorianChair implements Chair {
21     @Override
22     public void sitOn() {
23         System.out.println("Sitting on a Victorian chair.");
24     }
25 }
26
27 // ConcreteProduct: ModernSofa
28 class ModernSofa implements Sofa {
29     @Override
30     public void lieOn() {
31         System.out.println("Lying on a modern sofa.");
32     }
33 }
```

```
35 // ConcreteProduct: VictorianSofa
36 v class VictorianSofa implements Sofa {
37     @Override
38 v     public void lieOn() {
39         System.out.println("Lying on a Victorian sofa.");
40     }
41 }
42
43 // AbstractFactory: FurnitureFactory
44 v interface FurnitureFactory {
45     Chair createChair();
46     Sofa createSofa();
47 }
48
49 // ConcreteFactory: ModernFurnitureFactory
50 v class ModernFurnitureFactory implements FurnitureFactory {
51     @Override
52 v     public Chair createChair() {
53         return new ModernChair();
54     }
55
56     @Override
57 v     public Sofa createSofa() {
58         return new ModernSofa();
59     }
60 }
61
62 // ConcreteFactory: VictorianFurnitureFactory
63 v class VictorianFurnitureFactory implements FurnitureFactory {
64     @Override
65 v     public Chair createChair() {
66         return new VictorianChair();
67     }
68
69     @Override
70 v     public Sofa createSofa() {
71         return new VictorianSofa();
72     }
73 }
74
```



```

75 // Client
76 public class FurnitureStore {
77     private Chair chair;
78     private Sofa sofa;
79
80     public FurnitureStore(FurnitureFactory factory) {
81         chair = factory.createChair();
82         sofa = factory.createSofa();
83     }
84
85     public void displayFurniture() {
86         chair.sitOn();
87         sofa.lieOn();
88     }
89
90     public static void main(String[] args) {
91         FurnitureFactory modernFactory = new ModernFurnitureFactory();
92         FurnitureStore modernStore = new FurnitureStore(modernFactory);
93         modernStore.displayFurniture();
94
95         FurnitureFactory victorianFactory = new VictorianFurnitureFactory();
96         FurnitureStore victorianStore = new FurnitureStore(victorianFactory);
97         victorianStore.displayFurniture();
98     }
99 }
100

```

Q1	Why is access to non-static variables not allowed from static methods in OOP?	3
Q2	Which design pattern provides a way for object cloning? Explain a scenario where this pattern can be used.	3
Q3	"Talk to the interface, not to the concrete class" -- Explain this statement with appropriate reasoning.	3
Q4	Name two different patterns that are used to eliminate if else statement in a code? Give an example for each of these two patterns to explain how these patterns help eliminating if else?	6

a)

In object-oriented programming (OOP), **access to non-static variables is not allowed from static methods** because static methods belong to the **class itself**, while non-static variables (or instance variables) belong to **individual instances (objects) of the class**.

Reasons Why Non-Static Variables Cannot Be Accessed from Static Methods

1. **Static Methods Do Not Have Access to Instance-Specific Data**

- A static method operates at the **class level** and does not know which specific object's instance variable to access.
- Since instance variables are tied to objects, accessing them from a static method would require specifying which object they belong to.

2. **No Implicit `this` Reference in Static Methods**

- In instance methods, `this` refers to the current object, allowing access to instance variables.
- Static methods do not have a `this` reference because they are **not tied to any particular object**, making it impossible to access instance variables directly.

b)

Design Pattern for Object Cloning: Prototype Pattern

The **Prototype Pattern** is a **creational design pattern** that provides a mechanism for cloning objects instead of creating them from scratch. It allows an object to copy itself, reducing the overhead of expensive object creation, especially when instantiation is costly or complex.

Imagine you are working on a **drawing application** where users can create different shapes like **circles, squares, and rectangles**. Instead of creating a new shape from scratch each time, the application should allow users to **duplicate** an existing shape with the same properties (color, size, position).

For example, if a user **draws a red circle and wants a copy**, the system should **clone** the original circle instead of reinitializing all its properties manually.

Using the **Prototype Pattern**, we can **clone** the shape, keeping all attributes intact, which is more efficient than recreating it.